

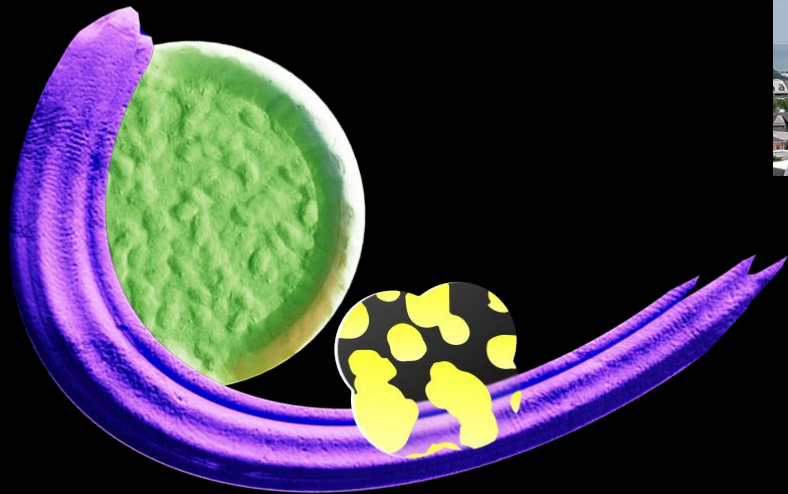
Отметьте студентов в журнале

Масштабируемость, отказоустойчивость и управление рисками

Михаил Каморин
Tech Lead



Speaker



Михаил Каморин
Tech Lead

16 лет в IT, 6 лет вёл
разные курсы в OTUS

Сегодня вы узнаете, как:

- 1 Разрабатывать масштабируемую архитектуру
- 2 Применять паттерны отказоустойчивости
- 3 Использовать SLI/SLO/SLA



План занятия

- 1 От одного узла к распределённой системе
- 2 Проектирование на случай непредвиденного
- 3 Моделирование поведения под нагрузкой
- 4 Управление качеством через SLI/SLO/SLA

I. Масштабируемость: от одного узла к распределенной системе

01 Типы масштабирования

02 Масштабирование источников данных



Вертикальное масштабирование

- **Просто** – код не требуется изменять
- **Быстро** – выделили более мощное железо и забыли
- **Дорого** – мощное железо непропорционально дороже
- **Есть предел** – нельзя бесконечно наращивать мощность

Горизонтальное масштабирование

- **Эластичность** – можно выделять ресурсы в зависимости от нагрузки
- **Нет предела** – всегда можно добавить ещё серверов
- **Сложно** – требует изменения архитектуры

Stateless-компоненты

- Каждый экземпляр сервиса независим от контекста
- Состояние выносится во внешние хранилища:
 - База данных
 - Кэш
 - Брокер сообщений

Балансировка нагрузки

- Равномерное распределение запросов между сервисами
- Возможность убирать узел из пула в случае отказа и возвращать обратно при восстановлении

Масштабирование источников данных

- **Репликация** – копирование данных на несколько серверов
- **Шардирование** – разделение данных на сегменты и помещение на несколько серверов
- **Кэширование** – хранение части данных в более быстром хранилище

Согласованность данных

- Сильная согласованность в распределённой системе часто недостижима
- Согласованность в конечном счёте
 - Окно несогласованности **ограничено**
 - Продолжительность окна согласованности заранее **неизвестна**
 - **Вероятностная природа** ответа на вопрос о согласованности

Вопрос для общего обсуждения

Почему вертикальное масштабирование хорошо на начальных этапах роста приложения, но затем перестаёт помогать?

2 минуты



Вопрос для общего обсуждения

Всегда ли стоит начинать с вертикального масштабирования? Почему?

2 минуты

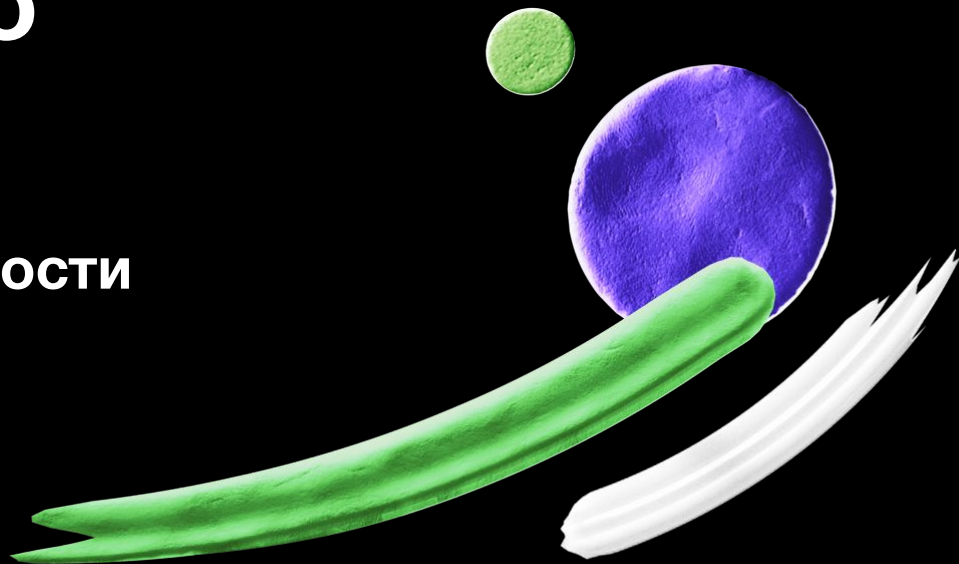


II. Отказоустойчивость: проектируем на случай непредвиденного

01 Классы сбоев

02 Паттерны отказоустойчивости

03 Репликация и кворум



Аппаратные сбои

- Отказ железа (дисковый массив, целый сервер)
- Часто происходят внезапно и выводят из строя целый узел

Пример: RAID-контроллер защищает от потери данных, но если падает весь сервер, то это вне его зоны ответственности

Сетевые сбои

- Разрыв соединений, высокая латентность, потеря пакетов
- Критичны для распределённых систем

Пример: сервис аутентификации не отвечает и запросы к API отваливаются по таймауту

Программные сбои

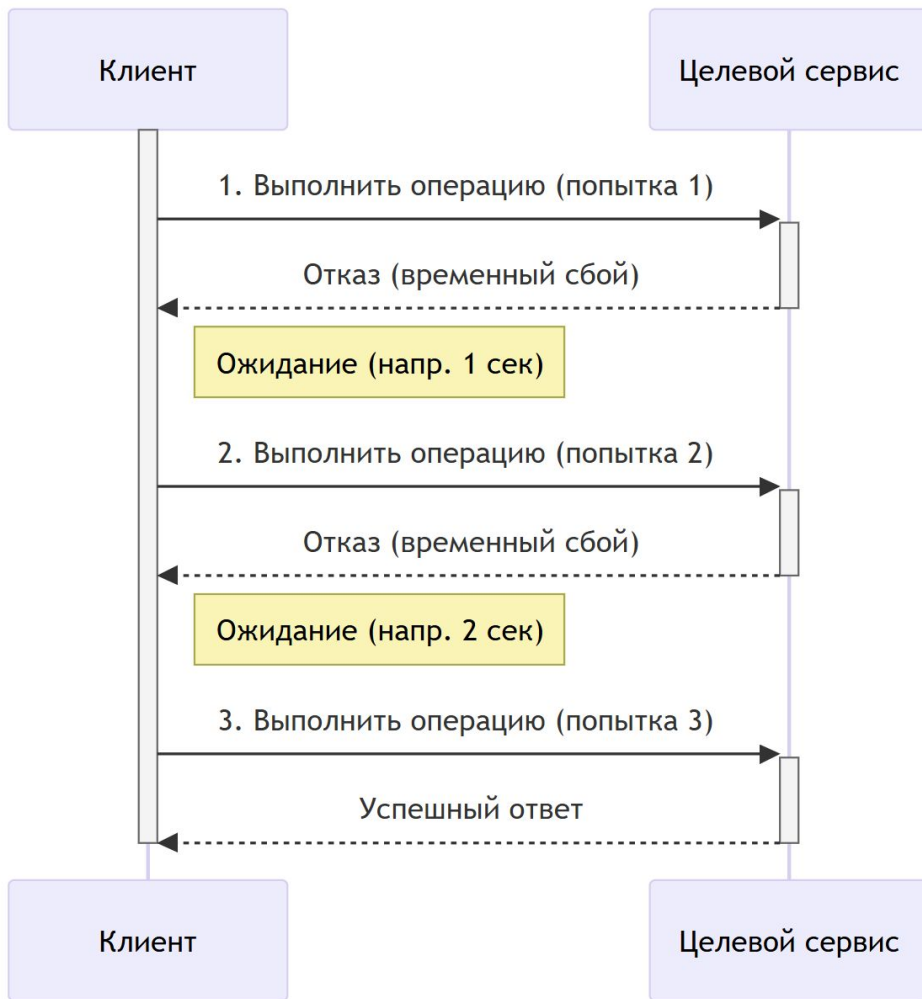
- Ошибки в коде, утечки памяти, зависания.
- Сюда же относятся баги в библиотеках и пакетах

Пример: бесконечный цикл в обработчике запроса приводит к зависанию

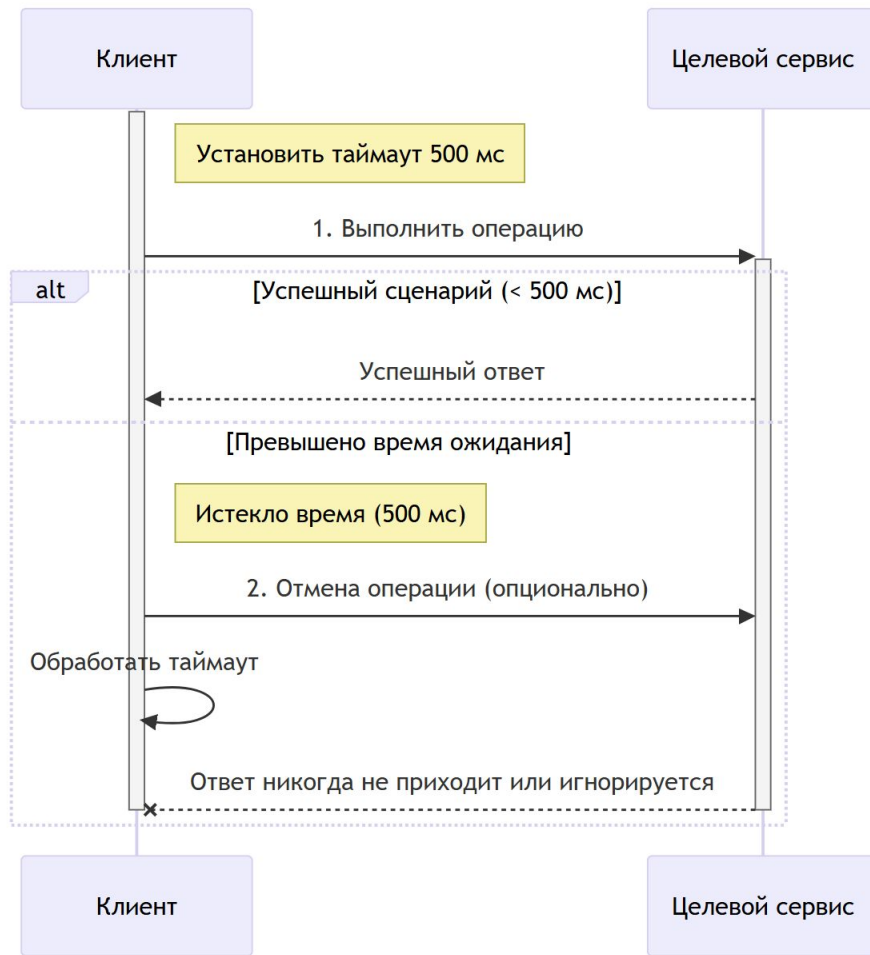
Внешние зависимости

- API сторонних сервисов, которые мы используем
- Если сторонний сервис падает, наша система должна как минимум корректно деградировать

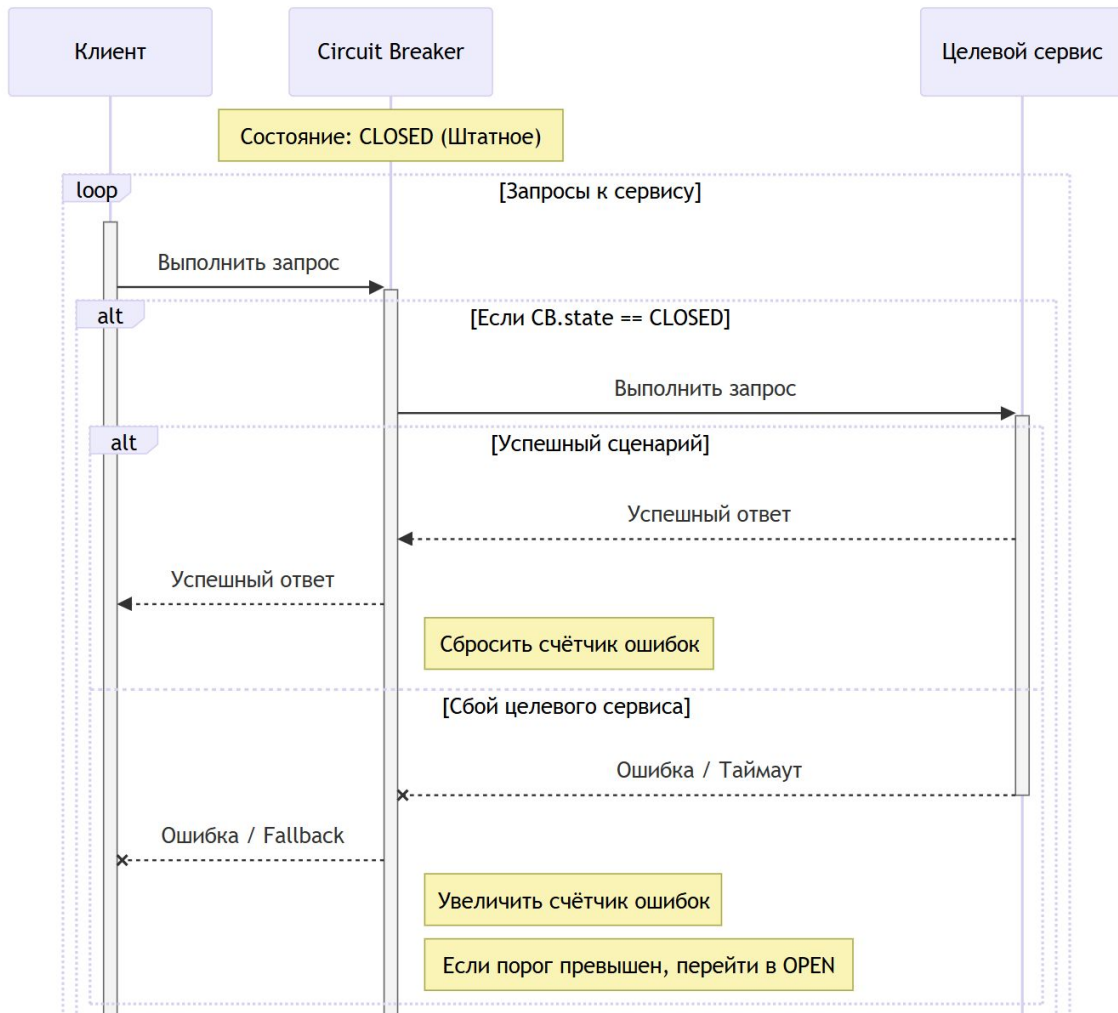
Retry



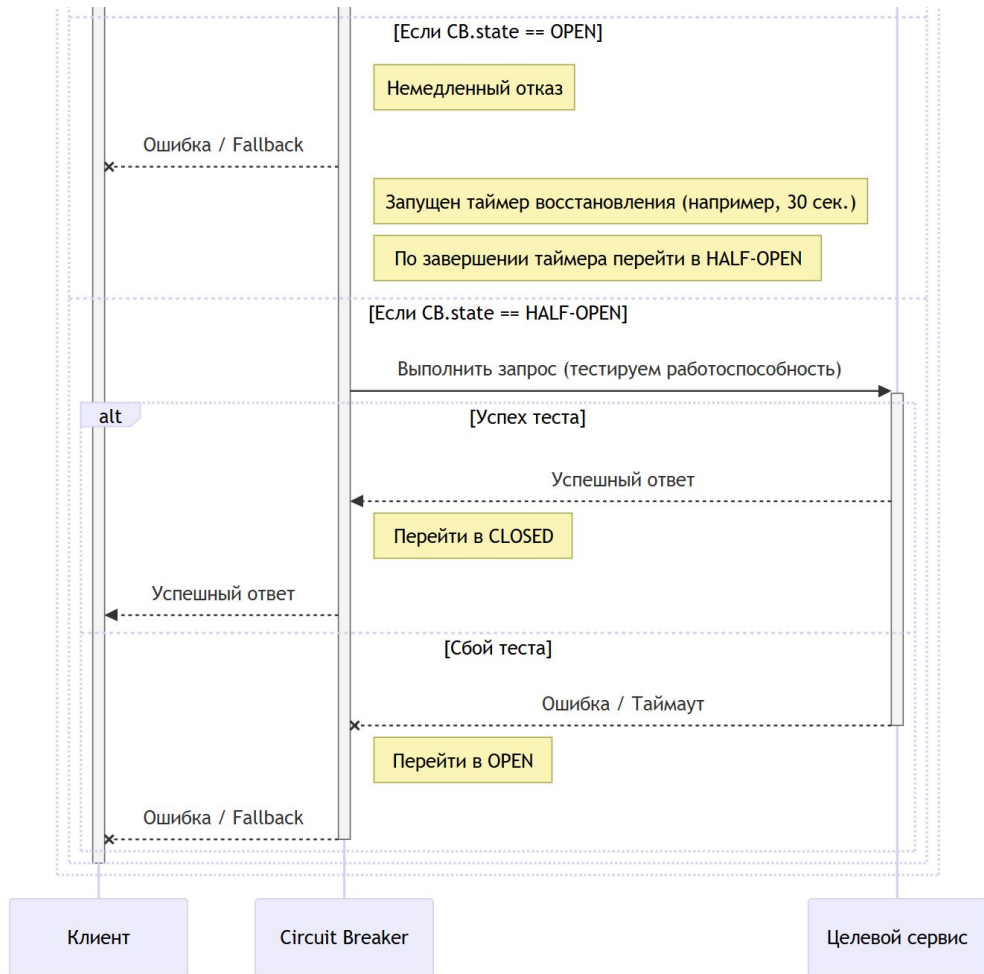
Timeout



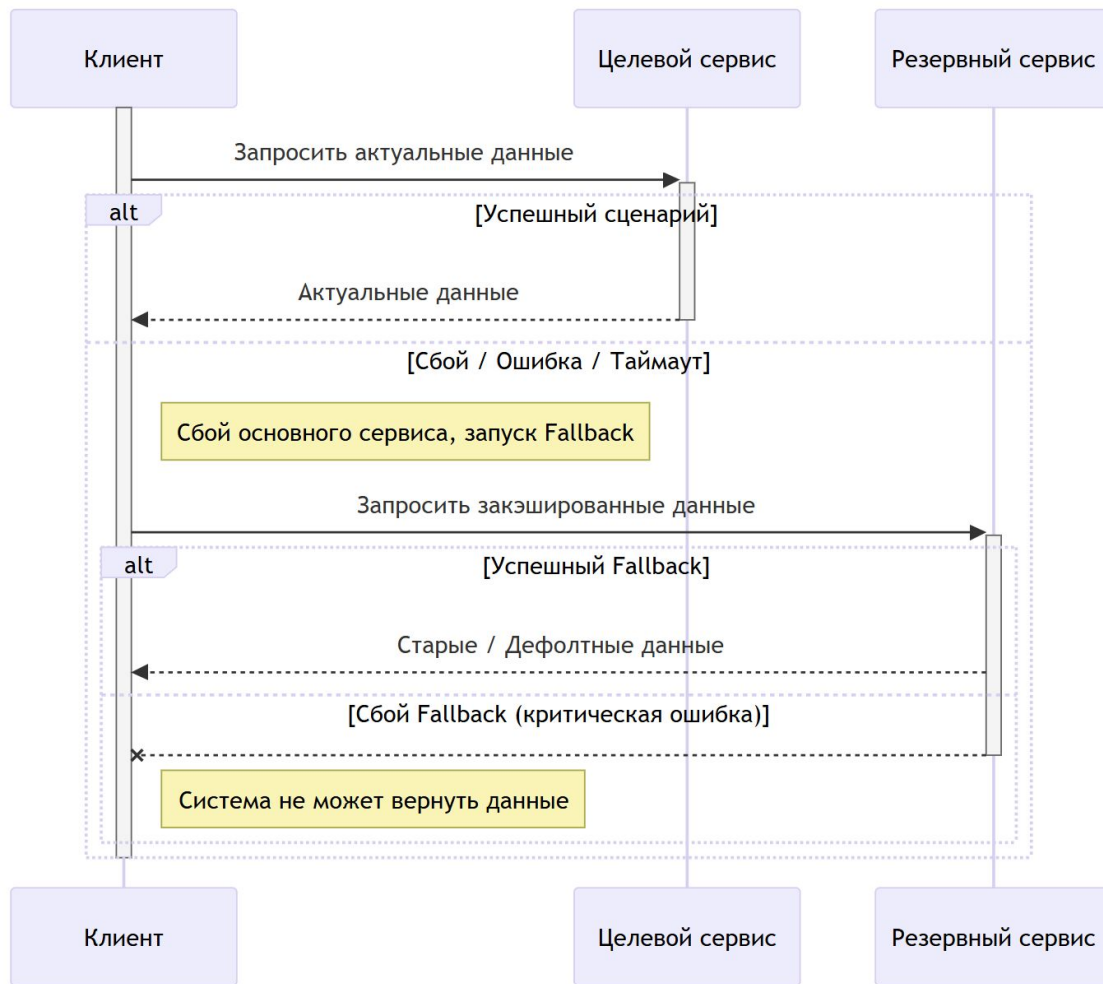
Circuit Breaker



Circuit Breaker



Fallback



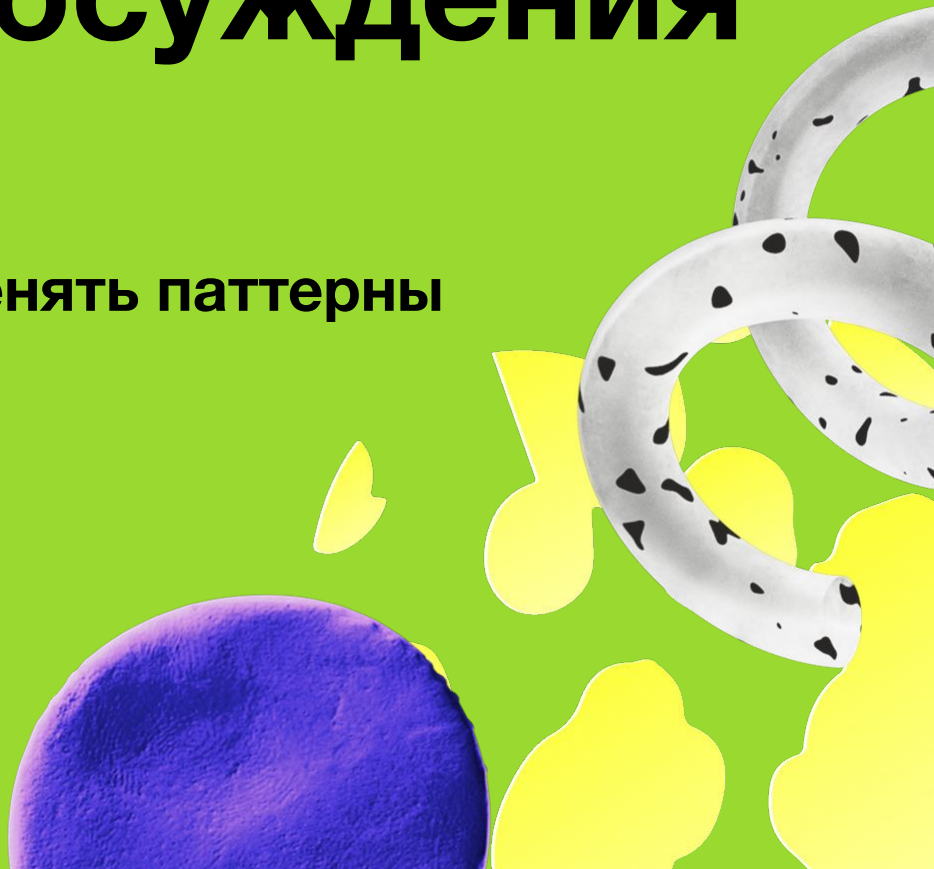
Репликация и кворум

- **Кворум** – число реплик, которые должны подтвердить операцию, чтобы она считалась успешной:
 - Кворум управляет “силой” согласованности
 - Чем выше согласованность, тем выше риск блокировки

Вопрос для обсуждения в парах

В каком порядке нужно применять паттерны отказоустойчивости?

3 минуты

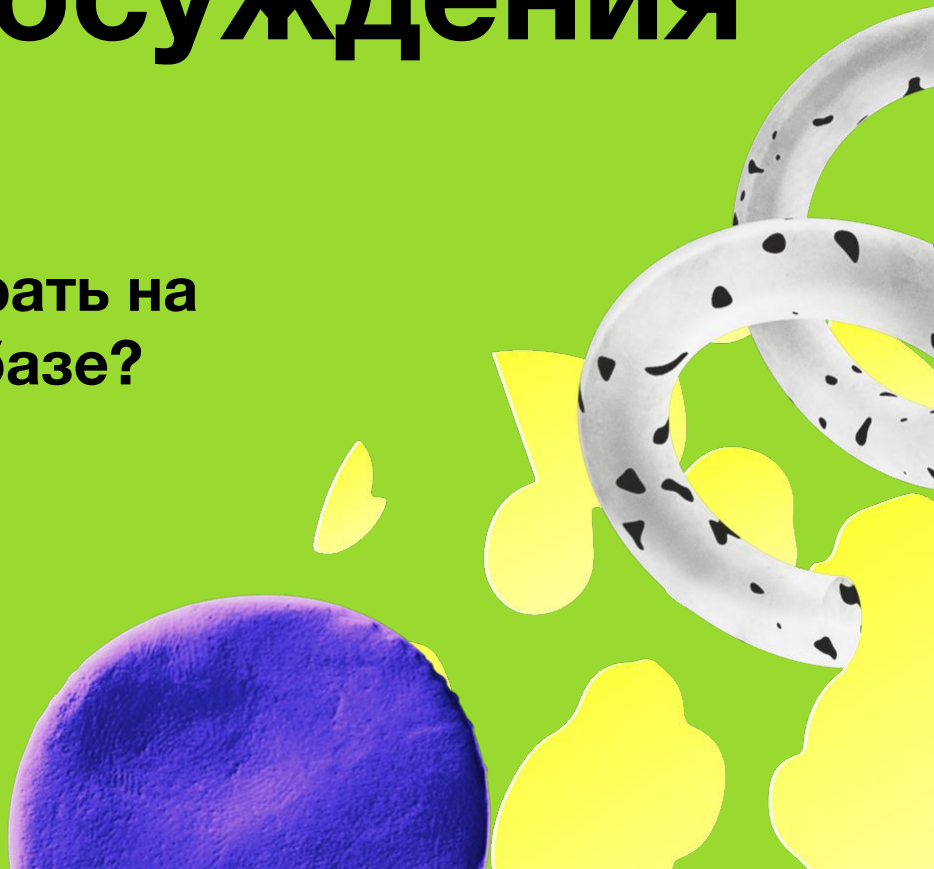


Вопрос для обсуждения в парах

Какой кворум правильно выбрать на
распределённой на 6 реплик базе?

Почему?

2 минуты

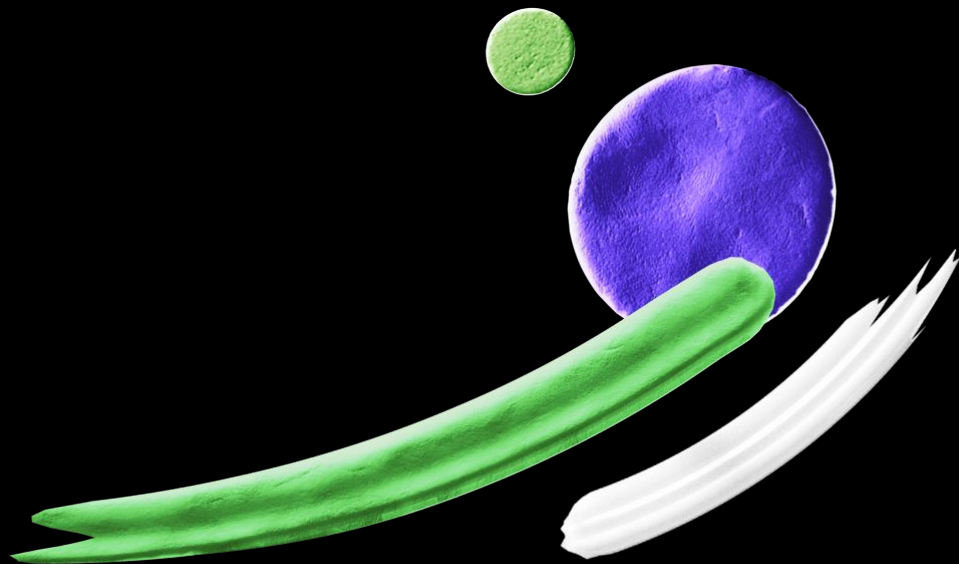


III. Моделирование поведения под нагрузкой

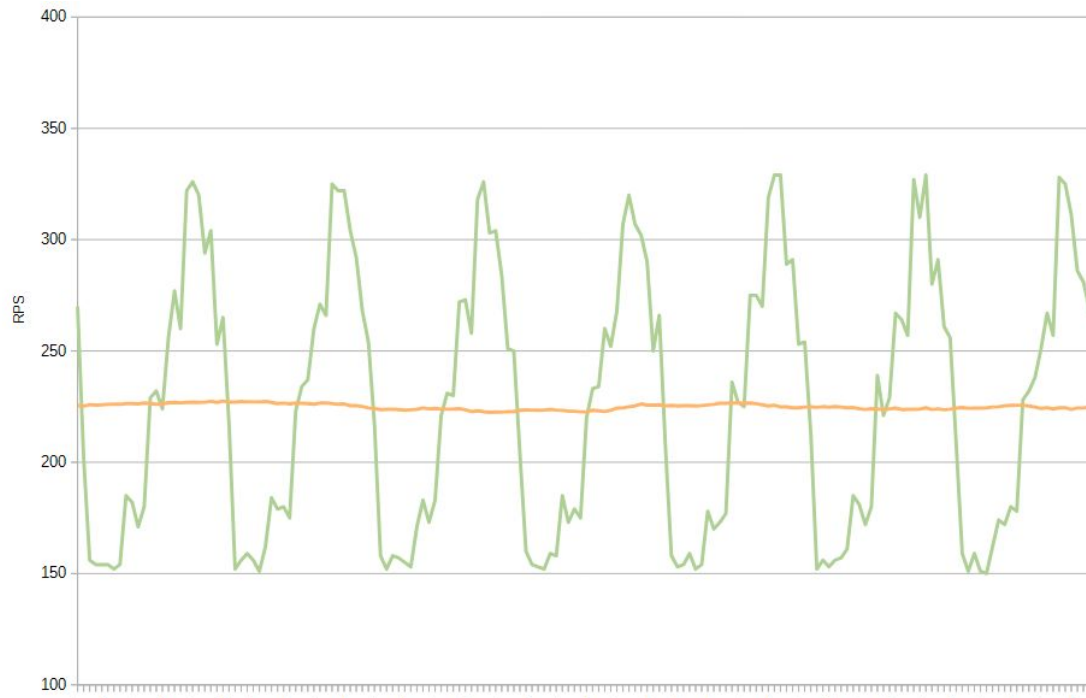
01 Типы нагрузки

02 Метрики нагрузки

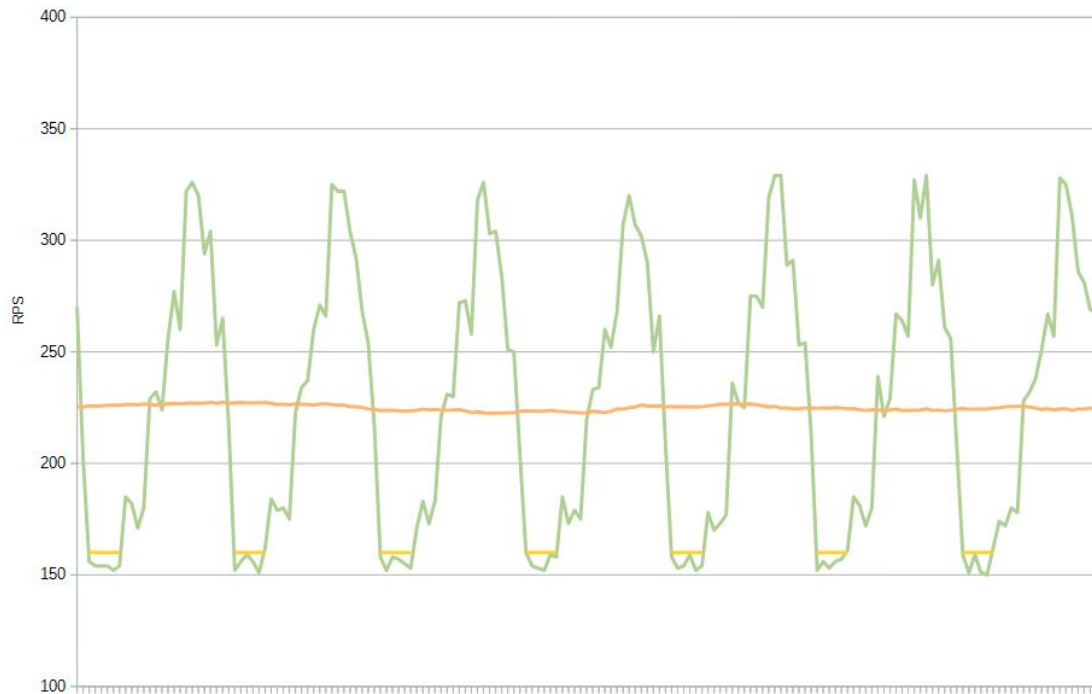
03 Тестирование на нагрузку



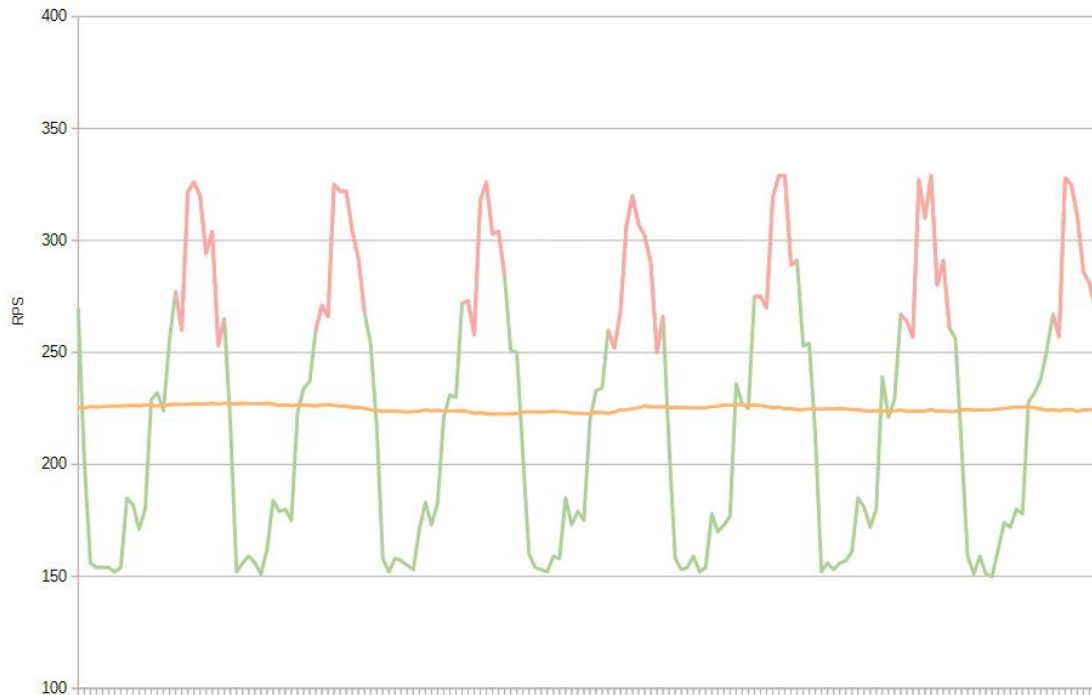
Регулярная нагрузка



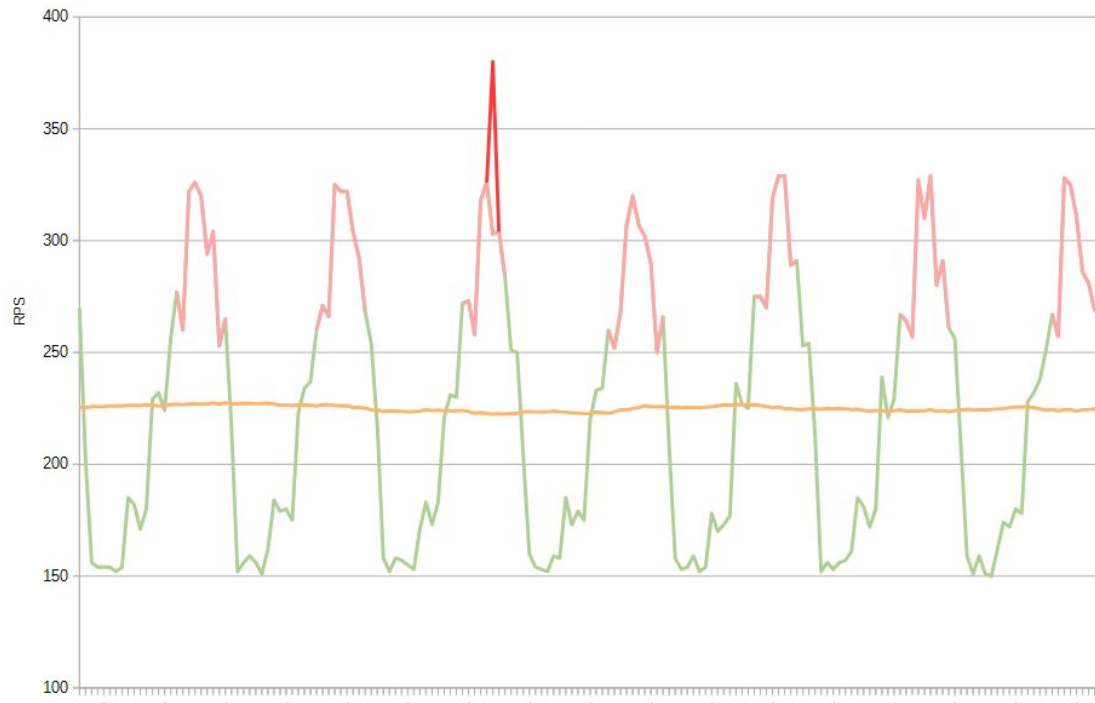
Фоновая нагрузка



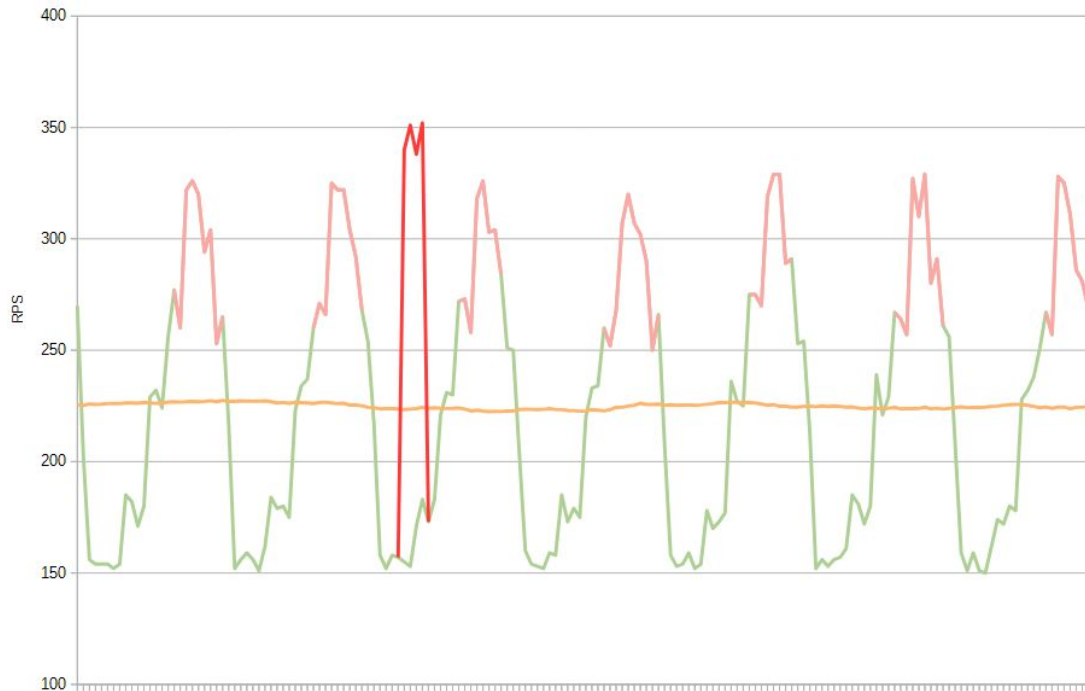
Пиковая нагрузка



Burst-нагрузка



Непредсказуемая нагрузка



Метрики нагрузки

- **RPS / QPS** – количество запросов в секунду
- **Latency** – задержка между запросом и ответом
- **Throughput** – сколько данных проходит через системы за единицу времени
- **Queue depth** – количество запросов в очереди на обработку

Тестирование на нагрузку

- **JMeter** – классический инструмент для нагрузочного тестирования веб-приложений
- **K6** – более современный инструмент, удобен для работы с JavaScript
- **Gatling** – Scala-библиотека для симуляции больших потоков

Методики нагрузочного тестирования

- **Стресс-тесты** – постоянное повышение нагрузки для поиска точки отказа
- **Spike-тесты** – резкие всплески нагрузки для проверки burst-устойчивости
- **Soak-тесты** – длительное воздействие обычной или повышенной нагрузки, применяется для выявления утечек ресурсов

Адаптивное масштабирование

- **Adaptive scaling** – добавление новых узлов при росте нагрузки

Пример: Kubernetes Horizontal Pod Autoscaler (HPA) добавляет новые pod'ы при росте нагрузки на CPU или росте RPS

Важно: узлы должны быть stateless, чтобы было доступно горизонтальное масштабирование

Плавная деградация

- **Graceful degradation** – при превышении возможностей системы она не падает полностью, а отключает тяжёлые функции

Пример: при высоком трафике видео-сервис снижает качество видео или приоритет кэша, но видео продолжает проигрываться

Предсказуемое планирование

- На основе анализа заранее готовим инфраструктуру под потенциальные пиковые нагрузки

Пример: добавляем серверы и реплики до “чёрной пятницы”, а не во время

Кейс

Кейс: Онлайн-магазин перед крупной распродажей (“чёрная пятница”).

Приведите примеры разных типов нагрузки и применения проектирования под пиковые сценарии

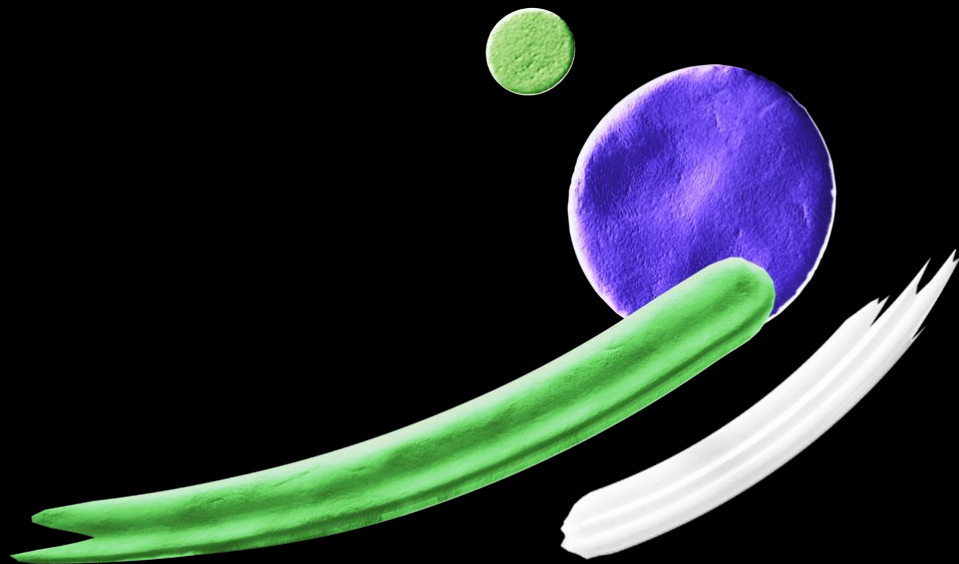
- **Формат:** делимся на три группы
- **Время:** 5 минут + по 1 минуте на мини-презентацию от каждой группы
- **Результат:** мини-презентация

IV. Архитектурное управление качеством через SLI/SLO/SLA

01 Что такое SLI/SLO/SLA

02 Примеры

03 Практики применения



Что значат эти аббревиатуры?

- **SLA** – ожидаемый уровень сервиса, наш контракт с заказчиком
- **SLO** – целевые показатели внутри SLA, помогают нам определить на что ориентироваться при проектировании системы
- **SLI** – что конкретно измеряем

Примеры SLI

- **Доступность (uptime)** – процент времени, когда система доступна для пользователей
- **Латентность (latency)** – время обработки запроса или транзакции
- **Доля ошибок (error rate)** – процент запросов с ошибочным ответом
- **Насыщенность (resource saturation)** – использование ключевых ресурсов (CPU, память, диск, сеть)

Пример

Параметр	Пример
SLA	Доступность $\geq 99.9\%$
SLO	99.95% транзакций успешно обрабатываются менее, чем за 500 мс
SLI	Error Rate ($< 0,05\%$) Latency (p9995 < 500 мс)

Практика применения

1. Выбираем критичные компоненты
2. Определяем метрики (SLI)
3. Задаём цели (SLO)
4. Контролируем архитектуру

Кейс

Кейс: Онлайн-сервис видеостриминга. SLA – 99% система доступна. Нужно предложить SLO и SLI согласно заданному SLA, а также какие архитектурные паттерны можно применить для соблюдения требований.

- **Формат:** делимся на три группы
- **Время:** 5 минут + по 1 минуте на мини-презентацию от каждой группы
- **Результат:** мини-презентация

Сегодня мы узнали, как:

- 1 Разрабатывать масштабируемую архитектуру
- 2 Применять паттерны отказоустойчивости
- 3 Использовать SLI/SLO/SLA



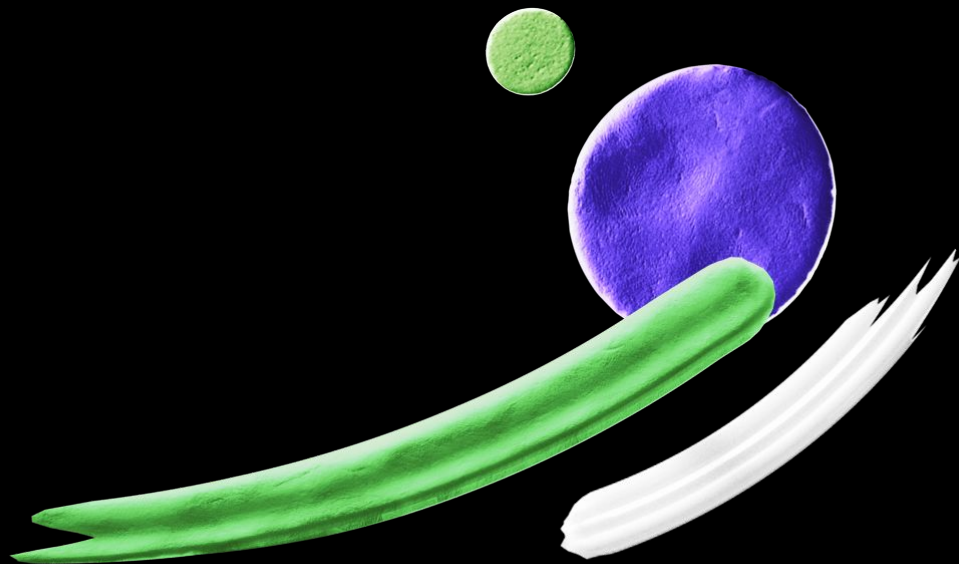
Вопросы?

Практика

01 задание

02 задание

03 задание



Условие

- Суть задания
- Формат
- Время
- Какой результат
- Что потребуется для выполнения